Theoretical Foundations of Proactive Interactive Agents

jiating.fang

Abstract

This paper establishes a theoretical framework for proactive interactive agents, addressing fundamental questions of what "proactive" means, when systems can initiate interaction, and why certain architectural constraints exist. We adopt emergence theory and abstraction-level analysis as a meta-theoretical stance, defining proactive interaction formally as the existence of system responses whose initiating user input is empty. The theory derives three original results: (1) the **Absence Detection Theorem**, proving that detecting the absence of expected events necessarily relies on time reference and cannot be achieved by pure event-driven approaches; (2) the **Triple Dilemma**, establishing that autonomy, low overhead, and real-time perception are mutually exclusive—only two of three objectives can be simultaneously satisfied; and (3) the **Triple Convergence Argument**, demonstrating that abandoning autonomy is the unique engineering-feasible solution jointly required by architectural constraints, user trust, and system security. These results form the theoretical foundation for the plugin-based proactive interaction architecture proposed in the companion whitepaper, establishing a closed theory-practice loop from philosophical grounding to engineering implementation.

Keywords:proactive interaction; emergence theory; triple dilemma; absence detection; agent architecture

Chapter 1: Meta-Principles - Theoretical Stance

1.1 Reductionism vs. Emergence (Borrowed)

Reductionism: Higher-level phenomena can be reduced to lower-level elements.
Emergence: Higher-level concepts have independent validity and cannot be simply reduced to lower levels (Anderson, 1972).
This Theory's Stance: Adopts emergence theory.

```
Applications:
- Proactivity holds at user layer, not negated by lower-level implementation
- Analogy: Concurrency holds at application layer, not negated by serial execution at bottom
- Analogy: Temperature holds at macro layer, not negated by molecular kinetics at bottom
```

1.2 Abstraction Levels Analytical Framework (Borrowed)

Level Spectrum:

```
User Layer (Experience) → Application Layer (Logic) → System Layer (Implementation) → Hardware Layer (Physical)
```

This Theory's Analytical Method:

• Define proactive interaction at user layer

• Analyze constraints at implementation layer
• Do not confuse judgments across layers

1.3 Methodological Rules

Rule 1: Before judging any conclusion, first clarify at which layer the judgment is made.
Rule 2: Architectural design is choosing appropriate abstraction level to solve problems, not reducing to the bottom.
Rule 3: Upper-layer definitions guide lower-layer design; lower-layer implementation does not negate upper-layer concepts.

Chapter 2: Definitions - Formalization of Proactive Interaction

2.1 Definition of Interaction

Definition 1 (Interaction): Information exchange process between system and user. Formalized as sequence:

```
S = {(u₁, r₁), (u₂, r₂), ..., (uₙ, rₙ)}
```

where $u_i \in U$ (user input), $r_i \in R$ (system response).

2.2 Definition of Proactive Interaction

Definition 2 (Proactive Interaction): There exists $r_i \in R$ such that $u_i = \emptyset$ (user did not issue command).
Definition 3 (Passive Interaction): $\forall r_i \in R$, there exists $u_i \in U$ such that r_i is a response to u_i.

2.3 Exclusion of Incorrect Definitions

Proposition 1: "Trigger script is written by user, therefore not proactive" is an incorrect definition.
Proof (by contradiction):

```
Assumption: Trigger is written by user → not proactive
Analogy: Alarm clock is set by user → alarm ringing is not proactive reminder

Contradiction:
- When alarm rings, user did not issue "report time" command
- By Definition 2, alarm ringing is proactive interaction

∴ Assumption is false
```

Judgment Standard: If the initiator of interaction is the system, proactive interaction definition is satisfied.

2.4 Necessary Conditions for Proactive Interaction

Proposition 2: Proactive interaction requires three core capabilities, none can be missing.

```
Perception: Acquire external state information
    ↓
Decision: Decide whether to trigger interaction
    ↓
Response: Execute interaction with user
```

Proof of Necessity:

• Without perception → no basis for triggering

• Without decision → random triggering or no triggering

• Without response → proactive interaction does not hold

Chapter 3: Core Concepts: Perception, Decision, Response

3.1 Perception

Definition 4 (Perception): System's capability to acquire external state information.

Perception Object Classification:

Table 1

Type	Description	Example
User State	User active/idle/focused	last_interaction_time
System State	CPU/memory/disk	cpu_temperature = 85°C
Task State	Complete/fail/progress	download.progress = 100%
Environment State	Time/location/network	current_time = 14:30

3.2 Decision

Definition 5 (Decision): System's capability to decide whether to trigger interaction based on perception results.

Formalization:

```
F(state, config) → {true, false}
```

3.3 Response

Definition 6 (Response): System's capability to interact with user after triggering.

Response Types:

Table 2

Type	Description	Example

Conversational	LLM generates reply	"CPU temperature too high, take care"
Task-oriented	Execute task then notify	"Cleaned 5 processes"
Silent	Execute in background without notification	Log rotation complete

Chapter 4: Key Concepts: Event and Absence

4.1 Event

Definition 7 (Event): State change that occurs at a specific point in time and can be captured by the system.
Formalization: e = (type, timestamp, payload)
Characteristics:

• Has clear occurrence time point

• Can be listened, captured, callback

• Is positive concept (something happens)

4.2 Absence

Definition 8 (Absence): State where expected event does not occur within specified time.
Formalization:

```
Let event set E = {e₁, e₂, ..., eₙ}
Let last_event_time = max{t(eᵢ) | eᵢ has occurred}

Absence holds ⇔current_time - last_event_time > threshold
```

Characteristics:

• No clear occurrence time point

• Cannot be directly listened

• Is negative concept (something does not happen)

• Detection must rely on time reference

4.3 Absence Detection Theorem (Original)

Theorem 1 (Absence Detection Theorem): Detecting absence must rely on time reference; cannot be achieved by pure event-driven approach.
Proof:

```
Premise 1: Event-driven requires events to listen to
Premise 2: Absence = no event occurs
Premise 3: "No event" is not an event
```

```
Inference:
- Absence cannot produce interrupt/callback/signal
- ∴ Absence cannot be detected by event-driven

Detection method:
- Record last_event_time
- Continuously compute current_time - last_event_time
- If difference > threshold → absence holds

∴ Absence detection must rely on time reference
```

Engineering Implication: User idle detection, sedentary reminder, and other absence scenarios cannot be achieved by pure event-driven approach.

Chapter 5: Structural Constraints: Triple Dilemma (Original)

5.1 Statement of Triple Dilemma

Theorem 2 (Triple Dilemma): In proactive interaction perception, the following three objectives cannot be simultaneously satisfied:

Table 3

Vertex	Definition	Ideal State
Autonomy	System autonomously decides what to perceive	Perceive whatever it wants
Low Overhead	Minimum resource consumption, no idle spinning	CPU 0%, reasoning cost 0
Real-time	State changes perceived immediately	Zero latency

5.2 Proof of Triple Dilemma

Proof (by case analysis):

Case 1: Autonomy + Low Overhead

```
Approach: Periodic polling
- Can perceive arbitrary state (Autonomy ✓)
- Longer period → lower overhead (Low Overhead ✓)
- But longer period → higher latency (Real-time ✗)
∴ Sacrifice real-time
```

Case 2: Low Overhead + Real-time

```
Approach: Event-driven
- Event perceived immediately (Real-time ✓)
- No idle spinning when no event (Low Overhead ✓)
- But can only perceive states with event sources (Autonomy ✗)
∴ Sacrifice autonomy
```

Case 3: Autonomy + Real-time

```
Approach: Continuous monitoring
- Can perceive arbitrary state (Autonomy ✓)
- State changes perceived immediately (Real-time ✓)
- But continuously occupies resources (Low Overhead ✗)
∴ Sacrifice low overhead
```

∴ Any two can be satisfied simultaneously, the third must be sacrificed.

Chapter 6: Architectural Decision: Triple Convergence (Original)

6.1 Triple Convergence Argument

Proposition 3: Abandoning autonomy is a joint requirement of engineering feasibility, user trust, and system security.

Table 4

Dimension	Argument	Conclusion
Engineering Constraints	Triple dilemma proves three cannot coexist	Must abandon one vertex
Trust Requirement	User needs to know what system perceives	Perception logic must be transparent
Security Requirement	System can only perceive authorized content	Perception boundary must be explicitly declared

∴ Triple convergence to same decision: Abandon autonomy, perception goals explicitly defined by humans.

6.2 Deep Meaning of Abandoning Autonomy

Proposition 4: Abandoning autonomy is not engineering compromise, but foundation of trust model.

Table 5

Challenge	Response
"Abandoning autonomy = capability degradation"	✗Wrong — it's the foundation of trust model
"Can be restored when technology matures"	□ Partially correct — can evolve, but current is optimal
"Do users really care?"	✓Empirical — privacy compliance requires explicit

	authorization

Chapter 7: From Constraints to Design: Mapping Framework (Original)

7.1 Constraint→Decision Mapping Table

Table 6

Constraint	Design Response	Implementation Mechanism
Perception goals must be predefined	Pluginization	`active/plugin/trigger` explicit declaration
Absence detection relies on time	Layered implementation	threshold (countdown) + interval (periodic)
Abandon autonomy for low overhead + real-time	trigger script lightweight judgment	100ms timeout, non-LLM
Different scenarios, different tradeoffs	Multi-mode triggering	periodic / idle (idle_once is a variant)

7.2 Formalization of Pluginization

Definition 9 (Plugin): Basic unit of proactive interaction, formalized as triple:

```
Plugin = (T, C, A)

Where:
T: trigger script, T: State → {true, false}
C: trigger mode configuration, C ∈{periodic, idle} (idle_once is a parameterized variant of idle)
A: interaction configuration, A → Prompt
```

Proposition 5: Pluginization transforms "perception must be predefined" into explicit architectural contract.

Contract Content:

Controllable: User knows what each plugin perceives

Auditable: Each plugin's trigger script is readable and reviewable

Shareable: Plugins can be distributed, reused, iterated in community

Chapter 8: Classification of Trigger Modes (Inductive Observation)

8.1 Two-Way Classification of Trigger Sources

Observation 1: Trigger sources of proactive interaction can be inductively classified into two categories.

Table 7

Source	Detection Method	Corresponding Mode	Example

Time elapsing	Timer	periodic	Check CPU every 60 seconds
Activity absence	Time difference calculation	idle (idle_once is a variant)	User no input for 5 minutes

Engineering Considerations:

• **periodic**: Covers states without event sources (CPU temperature, etc.) — relies on time elapsing

• **idle**: Covers user idle scenarios (absence detection, Theorem 1) — relies on time difference calculation

• **idle_once**: Variant of idle, controlled by triggered_count=1, triggers once per idle period

Note:

• Core modes: **periodic + idle**

• idle_once is a variant of idle, not independent mode

8.2 Extensibility Notes

Table 8

Extension Type	Description	Example
Intra-mode extension	New plugin instances	cpu_monitor → memory_monitor
Parameter extension	Adjust interval/threshold	60s → 300s
Logic extension	Modify trigger script	Temperature threshold 85°C → 90°C

Chapter 9: User Layer Verification

9.1 Consistency Verification

Table 9

Layer	Description	Consistent?
User Layer	User did not issue command → received response	✅Proactive interaction holds (Definition 2)
Implementation Layer	Polling, predefined, LLM passively called	✅Constraints satisfied (Theorem 1, 2)
Mapping Relation	Implementation constraints don't negate user layer definition	✅Meta-principle satisfied (Chapter 1)

9.2 Emergence of Proactivity

Proposition 6: Proactivity is a user-layer emergent concept; there is no "proactivity" concept at implementation layer.

Proof:

```
Implementation layer only has:
- Timer triggering
- Script execution
- Callback invocation

User layer emerges:
- "System proactively notified me"
- "Agent anticipated my needs"
- "Intelligent assistant"

∴ Proactivity is a user-layer emergent concept (Chapter 1 Meta-Principle)
```

Chapter 10: Conclusion and Future Evolution

10.1 Summary of Theoretical Contributions

Table 10

Type	Contribution	Chapter
Original Theorem	Absence Detection Theorem	Chapter 4
Original Theorem	Triple Dilemma	Chapter 5
Original Proposition	Triple Convergence Argument	Chapter 6
Original Methodology	Constraint→Decision Mapping Framework	Chapter 7
Original Formalization	Proactive interaction definition, plugin formalization	Chapter 2, 7
Inductive Observation	Two-way classification of trigger modes	Chapter 8
Borrowed Framework	Emergence theory, abstraction levels	Chapter 1

10.2 Future Evolution Path

Table 11

Stage	Perception Method	Decision Method	Description
Current (L1)	Plugin predefined	Script judgment	Engineering feasible
Mid-term (L1.5)	LLM-assisted recommendation	Script + ML	Semi-autonomous evolution
Long-term (L2)	Agent autonomously discovers	Agent planning	Fully autonomous exploration

10.3 Theory-Practice Closed Loop

Table 12

Document	Questions Answered	Position
Whitepaper	What & How	Engineering practice
Theoretical Foundations	Why & Why Not	Theoretical support

References

Anderson, P. W. (1972). More Is Different. Science.

Brooks, F. P. (1995). The Mythical Man-Month.

Shannon, C. E. (1948). A Mathematical Theory of Communication.

Turing, A. M. (1936). On Computable Numbers.

Citation & License

DOI: 10.5281/zenodo.19762639

Licensed under [Creative Commons Attribution 4.0 International (CC BY 4.0)](https://creativecommons.org/licenses/by/4.0/)